COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE 21S1
DATE PERFORMED:	07/16/2026	**DATE SUBMITTED:**	07/21/2026
NAME:	Kenji Baltazar	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4 **Hands-on Activity 2.1 Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

Run the code and show the output. Include the screenshot in table 2-1 followed by your observation

CODE

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //Constructor
        Student(std::string newName = "John Doe", int newAge = 18) {
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor was called" << std::endl;
        }
        //Deconstructor
        ~Student(){
            std::cout << "Destructor was called" << std::endl;
        }

        //Copy Constructor
        Student (const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }
```

```
26
27          //Display Attributes
28          void printDetails(){
29              std::cout << this->studentName << " " << this->studentAge << std::endl;
30          }
31
32      };
33
34  int main() {
35      Student student1("Kenji, 19");
36      Student student2(student1);
37      Student student3;
38      student3 = student2;
39
40      return 0;
41  }
```

OUTPUT

```
Constructor was called
Copy Constructor Called
Constructor was called
Destructor was called
Destructor was called
Destructor was called
```

OBSERVATION

The code works as follows:

- The libraries were first checked
- Then onto the int main where ***line 35*** is executed. It will also be placed in the object "**student 3**" having the **studentName** as "**Kenji, 19**"
- The constructor will then be executed which adds the "" at the object "**student 1**", at which will move "**Kenji, 19**" onto object "**student 1**" as well as constructing the **studentAge** "**18**". Thus, printing "**Constructor was called**".
- The copy constructor will then be executed which copies the **studentName** "**Kenji, 19**", displaying "**Copy Constructor Called**", as well as copying the **studentAge** "**18**" onto object "**student 1**"
- The constructor will be called again, filling up/ creating the **studentName** "**John Doe**" and creating **studentAge** "**18**" onto object "**student 3**". Thus, printing "**Constructor was called**".
- After all the lines were executed in the int main, it will return to 0, thus the deconstructor will be activated. At which, it will deconstruct 3 times and print "**Deconstructor was called**" 3 times.

Table 2-1. Initial Driver Program

Run the code with the modified driver function and show the output. Include the screenshot in table 2-2 followed by your observation.

CODE

```
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //Constructor
        Student(std::string newName = "John Doe", int newAge = 18) {
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor was called" << std::endl;
        }
        //Deconstructor
        ~Student(){
            std::cout << "Destructor was called" << std::endl;
        }

        //Copy Constructor
        Student (const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }
```

```
26
27          //Display Attributes
28          void printDetails(){
29              std::cout << this->studentName << " " << this->studentAge << std::endl;
30          }
31
32      };
33
34  int main() {
35      const size_t j = 5;
36
37      Student studentList[j] ={};
38      std::string namesList[j] = {"Acelajado", "Aguila", "Alvarez", "Baltazar", "Cabangon"};
39      int ageList[j] = {21, 18, 19, 19, 20};
40
41      return 0;
42  }
```

OUTPUT

```
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Destructor was called
Destructor was called
Destructor was called
Destructor was called
Destructor was called
```

OBSERVATION

The code works as follows:

- The libraries were first checked
- Then onto the int main where ***line 35*** is executed which initializes "**size_t j = 5**". Then ***line 37*** is executed adding "**John Doe**" to the **namesList** at ***index 0***.
- The constructor will then be executed which moves **studentName** "**John Doe**" from **namesList** onto ***array 0*** of the **studentList** and constructs **studentAge** "**18**". Thus, printing "**Constructor was called**".
- ***line 37*** will then be executed yet again to add another "**John Doe**" to the **namesList** at ***index 0*** then moving it to another array in the **studentList**. This will keep on looping until all 5 elements of array in the **studentList** are filled creating a total of 5 printed "**Constructor was called**".
- ***line 38*** will then be executed which fills up all elements in the array **namesList**, similarly, at ***line 39*** it will fill up all the elements in the array **ageList**.
- After returning to 0, the deconstructor will be activated. At which, it will deconstruct 5 times and print "**Deconstructor was called**" 5 times.

Table 2-2. Modified Driver Program with Student Lists

Run the code with the modified driver function and show the output. Include the screenshot in table 2-2 followed by your observation.

GENERAL CODE

```
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //Constructor
        Student(std::string newName = "John Doe", int newAge = 18) {
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor was called" << std::endl;
        }
        //Deconstructor
        ~Student(){
            std::cout <<"Destructor was called" <<std::endl;
        }

        //Copy Constructor
        Student (const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }
```

```
26
27          //Display Attributes
28          void printDetails(){
29              std::cout << this->studentName << " " << this->studentAge << std::endl;
30          }
31
32      };
33
34  int main() {
35      const size_t j = 5;
36
37      Student studentList[j] = {};
38      std::string namesList[j] = {"Acelajado", "Aguila", "Alvarez", "Baltazar", "Cabangon"};
39      int ageList[j] = {21, 18, 19, 19, 20};
40
41      for(int i=0; i<j; i++){ //LOOP A
42          Student *ptr = new Student(namesList[i], ageList[i]);
43          studentList[i] = *ptr;
44      }
45
46      for(int i=0; i<j; i++){ //LOOP B
47          studentList[i].printDetails();
48      }
49
50      return 0;
51  }
```

LOOP A - CODE

```
for(int i=0; i<j; i++){ //LOOP A
    Student *ptr = new Student(namesList[i], ageList[i]);
    studentList[i] = *ptr;
}
```

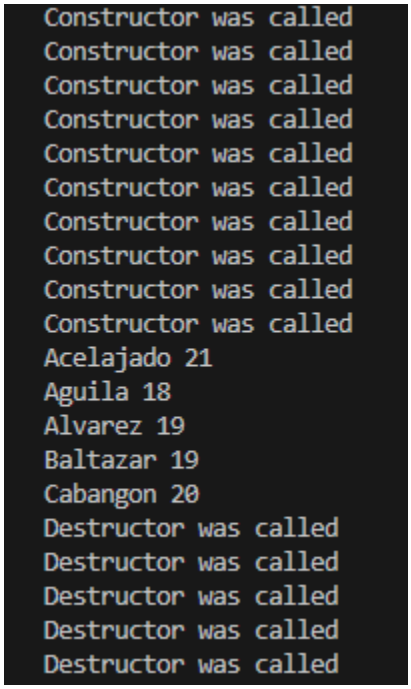
LOOP A - OBSERVATION

For each loop, **new Student(namesList[i], ageList[i])** makes a new object w/ name as well as age. Next, it calls the constructor, thus printing **"Constructor was called"**. Then **studentList[i] = *ptr** just copies those values into some of the existing array elements, and if there no new object, nothing prints.

LOOP B - CODE

```
for(int i=0; i<j; i++){ //LOOP B
    studentList[i].printDetails();
}
```

LOOP B - OBSERVATION
It just calls **printDetails()** on each of the 5 objects, printing their name as well as their age. Additionally, it only shows values already stored and the Copy constructor does not happen.

OUTPUT


```
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Acelajado 21
Aguila 18
Alvarez 19
Baltazar 19
Cabangon 20
Destructor was called
Destructor was called
Destructor was called
Destructor was called
Destructor was called
```

OUTPUT OBSERVATION
The output works as follows: - The first 5 of the**"Constructor was called"** from **studentList[j] = {}**. - Next is another 5 **"Constructor was called"** from **new Student** on the LoopA. And since copying values isn't making a new object, there is no Copy constructor that has been printed out. - The 5 names as well as age are from Loop B, which shows the data that is copied in Loop A. - Lastly, there is only 5 **"Destructor was called"** and it came from **studentList** being cleared at the end of the int main. I observed as well that the destructor does not match the constructor because there are 5 leaked new objects that are never cleaned up.

Table 2-3. Final Driver Program

Discuss what is done by loop A and loop B in table 2-3. Additionally, discuss the output and whether the functions are working as intended. If any corrections were made, further provide your modification and analysis in table 2-4

MODIFICATIONS

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //Constructor
        Student(std::string newName = "John Doe", int newAge = 18) {
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout << "Constructor was called" << std::endl;
        }
        //Deconstructor
        ~Student(){
            std::cout <<"Destructor was called" <<std::endl;
        }

        //Copy Constructor
        Student (const Student &copyStudent){
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }
```

```
26
27          //Display Attributes
28          void printDetails(){
29              std::cout << this->studentName << " " << this->studentAge << std::endl;
30          }
31
32      };
33
34  int main() {
35      const size_t j = 5;
36
37      Student studentList[j] = {};
38      std::string namesList[j] = {"Acelajado", "Aguila", "Alvarez", "Baltazar", "Cabangon"};
39      int ageList[j] = {21, 18, 19, 19, 20};
40
41      for(int i=0; i<j; i++){ //LOOP A
42          Student *ptr = new Student(namesList[i], ageList[i]);
43          studentList[i] = *ptr;
44          delete ptr; //THE ADDED ONE!
45      }
46
47      for(int i=0; i<j; i++){ //LOOP B
48          studentList[i].printDetails();
49      }
50
51      return 0;
52  }
```

OUTPUT

```
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Destructor was called
Constructor was called
Destructor was called
Constructor was called
Destructor was called
Constructor was called
Destructor was called
Constructor was called
Destructor was called
Acelajado 21
Aguila 18
Alvarez 19
Baltazar 19
Cabangon 20
Destructor was called
Destructor was called
Destructor was called
Destructor was called
Destructor was called
```

OBSERVATION
As a continued observation for Table 2-3, what happened there was that the destructor does not match the number of constructor. So, I experimented a bit placing **delete ptr** on loop A because I know that it is a for loop that call for constructor, thus the destructor will now appear on loop A right after each object has been created. This solves the problem of the memory leak as the destructor now matches the constructor.

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

CODE

```
4   class Fruit {
5   private:
6       std::string fruitName;
7       float moneyEach;
8       int howManyPls;
9
10  public:
11      //makes a new fruit thingy
12      Fruit(std::string n = "SomeFruit", double p = 0, int q = 0) {
13          fruitName = n;
14          moneyEach = p;
15          howManyPls = q;
16          std::cout << "New Fruit Constructed!" << std::endl;
17      }
18      //removes the fruit
19      ~Fruit() {
20          std::cout << "Deconstructed." << std::endl;
21      }
22      //this one copies the fruit
23      Fruit(Fruit& other) {
24          fruitName = other.fruitName;
25          moneyEach = other.moneyEach;
26          howManyPls = other.howManyPls;
27          std::cout << "Copied the fruit." << std::endl;
28      }
```

```
29      //copies it again but different
30      Fruit& operator=(Fruit& other) {
31          std::cout << "= copied the fruit again" << std::endl;
32          if (this != &other) {
33              fruitName = other.fruitName;
34              moneyEach = other.moneyEach;
35              howManyPls = other.howManyPls;
36          }
37          return *this;
38      }
39
40      double doMath(){
41          return moneyEach * howManyPls; // price times how many
42      }
43
44      std::string whatsMyName(){
45          return fruitName;
46      }
47
48      void showMe(){
49          std::cout << "[Fruit] " << fruitName << " - PHP " << moneyEach
50                    << " x " << howManyPls << " = PHP " << doMath() << std::endl;
51      }
52  };
```

```cpp
class Vegetable {
private:
    std::string veggieName;
    float moneyEach;
    int howManyPls;

public:
    Vegetable(std::string n = "SomeVeggie", double p = 0, int q = 0) {
        veggieName = n;
        moneyEach = p;
        howManyPls = q;
        std::cout << "New Veggie Constructed!" << std::endl;
    }
    ~Vegetable() {
        std::cout << "Deconstructed." << std::endl;
    }
    Vegetable(const Vegetable& other) {
        veggieName = other.veggieName;
        moneyEach = other.moneyEach;
        howManyPls = other.howManyPls;
        std::cout << "Copied the veggie." << std::endl;
    }
    Vegetable& operator=(Vegetable& other) {
        std::cout << "= copied the veggie again" << std::endl;
        if (this != &other) {
            veggieName = other.veggieName;
            moneyEach = other.moneyEach;
            howManyPls = other.howManyPls;
        }
        return *this;
    }
```

OBSERVATION

This problem 1 was where I created my classes, which has the BIG 3 as well as the constructor. I based off my code on the previous activity above. I also observed in this that I just basically did at veggie class what I first done to my fruit class, meaning that once I got a grasp to how class works, it just works the same or more like I can get it.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

CODE

```
113     int fruitCount = 2;
114     Fruit** fruitBasket = new Fruit*[fruitCount];
115     fruitBasket[0] = new Fruit("Apple", 10, 7);
116     fruitBasket[1] = new Fruit("Banana", 10, 8);
117
118     int veggieCount = 2;
119     Vegetable** veggieBasket = new Vegetable*[veggieCount];
120     veggieBasket[0] = new Vegetable("Broccoli", 60, 12);
121     veggieBasket[1] = new Vegetable("Lettuce", 50, 10);
122
123     std::cout << "\nGrocery List" << std::endl;
124     for (int i = 0; i < fruitCount; i++) {
125         fruitBasket[i]->showMe();
126     }
127     for (int i = 0; i < veggieCount; i++) {
128         veggieBasket[i]->showMe();
129     }
```

OBSERVATION

So in this code, it is just basically making a fruit basket which is filling up the array with fruit and veggies. I also observed that since fruit and veggie are different class, they cannot share the same array.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

CODE

```
101 double TotalSum(Fruit** myFruits, int fruitCount, Vegetable** myVeggies, int veggieCount) {
102     double bigTotal = 0;
103     for (int i = 0; i < fruitCount; i++) {
104         bigTotal = bigTotal + myFruits[i]->doMath();
105     }
106     for (int j = 0; j < veggieCount; j++) {
107         bigTotal = bigTotal + myVeggies[j]->doMath();
108     }
109     return bigTotal;
110 }
```

OBSERVATION

For this code, I just made a function to just add the sum for all fruits and veggies in the grocery list. Using the for loop to keep on running until all fruits and veggies are added in the bigtotal.

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

CODE

```
134     int byeIndex = -1;
135     for (int i = 0; i < veggieCount; i++) {
136         if (veggieBasket[i]->whatsMyName() == "Lettuce") {
137             byeIndex = i;
138             break;
139         }
140     }
141
142     if (byeIndex != -1) {
143         delete veggieBasket[byeIndex]; // free the memory
144
145         Vegetable** smolerBasket = new Vegetable*[veggieCount - 1];
146         int k = 0;
147         for (int i = 0; i < veggieCount; i++) {
148             if (i != byeIndex) {
149                 smolerBasket[k] = veggieBasket[i];
150                 k++;
151             }
152         }
153         delete[] veggieBasket;
154         veggieBasket = smolerBasket;
155         veggieCount = veggieCount - 1;
156     }
```

OBSERVATION

So this is the part that I was really confused about because removing an item from a dynamic array isn't as simple as just deleting one slot the arrays are fixed sized, because I need to make an entirely new, smaller array and the old one released needs to happen as well.

Conclusion: Overall, I learned the most to how to implement the BIG 3 as well as the constructor. Additionally, I learned the implementation for the class, because struct is the one I am more familiar about so this HOA helped me to really grasp the idea for class and the private and public functions in it. The procedure was fairly understandable as I was guided thoroughly by the manual, implementing constructor, destructor, copy constructor, and copy assignment operator left and right as well as learning the importance of each. The supplementary really drained me as I cannot understand much to what I was doing at around problem 4. I think I did well at the procedure but did badly in supplementary as I need to improve my own skills and understanding for each and every concept in this activity.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude, ChatGPT, Copilot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Error checkings and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to help me identify and troubleshoot my errors as well as assist me whenever I am stuck or forgot to code a part of a code. AI did not replace my original idea as it was based on the given problem as well as the completion of my work and the analysis was based off my own understanding.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.